ZetaERP — Manual Bug-Testing SOP

Part 5 — Inventory (Products, Stock, Warehouses, Transfers, Adjustments, Batches/Serials, Forecast)

Document	QA-SOP-05 — Inventory
Product	ZetaERP / Zenith (multi-tenant POS / ERP)
Environment under test	Production — https://erp.zetaclantech.com
Scope of this part	The **Inventory** module: Dashboard, Product Catalog, Catalogues, Warehouses/Facilities, Receiving (Purchase→stock), Stock Levels, Opening Stock, Stock Adjustments, Stock Transfers (inter-branch + e-Way Bill/IGST), Batch & Serial tracking, Expiry Risk, Demand Forecast & reorder alerts, Gate Passes — and the stock-movement integrations with **POS**, **Purchase** and **Sales**. (Accounting valuation ledgers and the Purchase order lifecycle itself are separate parts.)
Audience	Manual QA tester (no code knowledge assumed)
Document owner	Platform / Solo-dev
Status	Active
Last updated	2026-07-01

CONTENTS

1. Purpose
2. Critical rules for the tester (read before starting)
3. Test environment & preconditions
4. Test data
5. Conventions & severity
6. Flow map (stock lifecycle)
7. INVENTORY DASHBOARD
8. PRODUCT CATALOG & CATALOGUES
   - 8.1 Products
   - 8.2 Catalogues (categories)
9. WAREHOUSES & FACILITIES
10. OPENING STOCK & STOCK LEVELS
11. RECEIVING (Purchase → stock)
12. STOCK ADJUSTMENTS
13. STOCK TRANSFERS (inter-branch)
14. BATCH & SERIAL TRACKING
15. EXPIRY RISK
16. DEMAND FORECAST & REORDER ALERTS
17. GATE PASSES
18. INTEGRATION — stock movement end-to-end
19. MULTI-TENANT ISOLATION (high priority)
20. UI / ACCESSIBILITY / SECURITY
21. Cross-flow / regression & defect template
22. Areas to probe hardest (attention map)
23. Teardown (mandatory)

1. Purpose

This Standard Operating Procedure tells a tester **exactly how to manually test the Inventory module end-to-end (A→Z)** — the full **stock lifecycle**: create a product → set opening stock → receive against a purchase order → move stock between warehouses → adjust for physical count / damage / expiry → sell at POS (stock depletes) → watch reorder & demand alerts fire → track batches, serials and expiring goods → issue and close gate passes.

It covers the **edge cases** (negative stock, untracked products, zero-cost valuation, partial/over receipts, same-facility transfers, expired-stock movement), the **multi-tenant isolation** cases, the **security** cases, and the **visual/UI** cases, and **how to record a defect** when behaviour differs from the *Expected Result*.

> Stock numbers drive **what a cashier can sell**, **what the books are worth**, and **when the system tells you to reorder**. A wrong quantity is not cosmetic — treat quantity/valuation defects seriously.

2. Critical rules for the tester (read before starting)

> ⚠️ **This is a LIVE commercial system. Stock changes are real and immediately affect POS availability and stock valuation.**

1. **NEVER adjust, transfer, receive, or write off stock** on the live flagship tenant **MARUTHI (org `100000`)** or any live tenant. Every adjustment/transfer/receipt **mutates real on-hand quantities** and writes an `AssetDetail` ledger row. Do all create/receive/adjust/transfer testing on a **throwaway tenant** you control, with **throwaway products/facilities**.
2. ⚠️ **Known behaviour — there is no negative-stock guard.** The core stock service (`adjust#AssetOnHand`) does **not** stop on-hand going below zero. Transfers, adjustments and (mis-configured) POS sales can drive a quantity negative. Several cases below deliberately test this — do them **only on throwaway data**.
3. ⚠️ **Suspected isolation gap — Expiry Risk.** The **Expiry Risk** dashboard may read expiring stock **without an organization filter** (see §15/§19). Treat any cross-tenant leak here as **Critical** and verify carefully.
4. **Never fabricate** legal/tax data — GSTIN (facility/e-Way Bill), HSN/SAC codes, batch/lot numbers of real products, supplier PANs. Use clearly-fake QA values on a throwaway tenant.
5. **Multi-tenant isolation is a key surface** (§19). A tenant must never see or mutate another tenant's products, stock, facilities, transfers, adjustments, serials or alerts. Any cross-tenant read **or write** is **Critical**.
6. **Clean up every test artefact** you create — products, facilities, opening-stock, adjustments, transfers, gate passes, serials — or hand the list to the dev. Note that applied adjustments and shipped transfers **cannot be "un-applied"**; reverse them with an equal-and-opposite adjustment on throwaway data.
7. **Don't** run the **Demand Forecast "Run Cycle"** button on a live tenant casually — it can generate alerts and **push notifications to every user in the org**. Use a throwaway tenant.

3. Test environment & preconditions

Open from the app menu → **Inventory**. Screen URLs (`/apps/zenith/Inventory/…` — note the capital **I**):

Area	Screen (URL name)	Purpose
Dashboard	`InventoryDashboard`	KPIs: products, stock value, units, low-stock, expiry risk
Product Catalog	`FindProduct`, `EditProduct?productId=`	Create/list/edit products; opening stock; barcode
Catalogues	`CreateCatalogue`	Product categories (POS grid tabs)
Warehouses	`FindFacility`, `EditFacility?facilityId=`	Facilities/warehouses + GSTIN for e-Way Bill
Receiving	`FindReceipt`, `ReceiveInventory?orderId=`	Receive stock against a purchase order (creates Asset/batch)
Stock Levels	`StockLevels`	On-hand (QOH) / available-to-promise (ATP) / value per facility
Stock Transfers	`StockTransfers`, `EditStockTransfer?stockTransferId=`	Move stock between facilities; e-Way Bill; inter-state IGST
Stock Adjustments	`StockAdjustments`, `EditStockAdjustment?stockAdjustmentId=`	Physical count / damage / expiry / write-off / opening

Area	Screen (URL name)	Purpose
Serial Numbers	`SerialNumbers`	Register & track serials through their lifecycle
Demand Forecast	`DemandForecast`	Forecasts + reorder / demand-shift alerts
Expiry Risk	`ExpiryDashboard`	Stock expiring within ~90 days
Gate Passes	`GatePasses`	Returnable / non-returnable goods gate passes

Log in as a tenant user so an **active organization** scopes every screen. For isolation tests you need **two tenants**. Set up on the throwaway tenant: **2 facilities** (ideally one with a *different* GST state code, for inter-state transfer), **2–3 products** (one with **MRP + HSN + GST**, one **batch-tracked**, one **serial-tracked**), and a **purchase order** you can receive against.

Browsers: full pass on Desktop Chrome; spot-check Edge/Safari and a phone. **Tools:** DevTools (Console/Network), a calculator (to verify quantities, valuation and reorder math), and access to the **POS Terminal** and **Purchase** modules for the integration flows.

---

4. Test data

Label	Use
Throwaway tenant	Non-MARUTHI org for all stock mutations
`WH_A` / `WH_B`	Two facilities/warehouses; **same** GST state code
`WH_OTHER`	A facility with a **different** GST state code + a (fake) 15-char GSTIN → inter-state transfer
`PROD_TRACK`	Product with MRP, HSN, GST, **opening stock entered** (`inventoryTracked = Y`)
`PROD_UNTRACK`	Product with **no opening stock** (`inventoryTracked` not set)
`PROD_BATCH`	Product with **Batch tracking = Y**, a batch number + expiry date
`PROD_SERIAL`	Product with **Serial tracking = Y**
`PROD_EXPSOON`	A batch with `expiryDate` within ~90 days (for Expiry Risk)
Fake GSTIN	`29ABCDE1234F1Z5` -style dummy (throwaway only)
Quantities to verify	e.g. opening 100 → sell 3 at POS → expect 97; transfer 20 A→B → A=77, B=20

---

5. Conventions & severity

Case Type: `FUNC` · `EDGE` · `CALC` (quantity/valuation/reorder math) · `UI` · `A11Y` · `SEC` · `ISO` · `PERF` .

Sev	Meaning	Example
S1 — Critical	Cross-tenant read/write of stock; on-hand corrupted (double-deplete, phantom stock); wrong IGST on a branch transfer; data loss	Tenant A sees tenant B's expiring stock; ship depletes stock twice
S2 — High	A stock flow broken or a quantity clearly wrong	Receive doesn't add stock; POS sale doesn't deplete a tracked product; transfer received but destination unchanged
S3 — Medium	Wrong/loose behaviour, limited impact	Negative stock accepted; same-facility transfer allowed; no validation on quantity
S4 — Low	Cosmetic / minor UX	Badge colour wrong; column misaligned; value formatting

Logging a defect (template §21): Case ID, steps, **expected vs actual quantities/values** (show the arithmetic), tenant, facility/product ids, browser, screenshot, Console/Network capture. For CALC and ISO cases, evidence is **mandatory**.

---

6. Flow map (stock lifecycle)

```
PRODUCT ──opening stock──►  ON-HAND (Asset: quantityOnHandTotal / availableToPromise)
 (inventoryTracked=Y)            │
                                 ├──◄── RECEIVE against Purchase Order (creates Asset/batch, +qty)
                                 ├──►── STOCK TRANSFER  WH_A ──ship(-)──► in transit ──receive(+)──► WH_B
                                 │           (inter-state / >₹50k → e-Way Bill; diff GSTIN → IGST invoice)
                                 ├──►── STOCK ADJUSTMENT  (PhysicalCount/Damage/Expiry/WriteOff/Opening)
                                 │           diff = actual − system  →  apply → on-hand ±diff
                                 ├──►── POS SALE  (-qty, ONLY if product.inventoryTracked = Y)
                                 └──►── DEMAND FORECAST → reorder / demand-shift ALERTS

Every mutation writes an AssetDetail ledger row.  Reorder point = avgDailyDemand×leadTime + safetyStock,
safetyStock = Z(serviceLevel) × σ × √leadTime.   Batch = Asset.batchNumber/expiryDate;  Serial = ProductSerialNumber.
```

---

7. INVENTORY DASHBOARD

Screen: `InventoryDashboard`. Total products, total stock value, units in hand, low-stock count, expiry-risk count; links to Stock Levels, Expiry, Forecast, Adjustments, Transfers.

ID	Type	Title	Steps	Expected result	Sev
IV-001	FUNC	Dashboard renders	Open the dashboard	All KPI cards + any charts render; no console errors	S3
IV-002	FUNC	Figures org-scoped & correct	Cross-check "Total products" / "Units in hand" / "Stock value" against Product Catalog + Stock Levels	Numbers match **your** org only	S2
IV-003	CALC	Stock value math	Value = Σ(on-hand × cost/valuation) for your org	Matches Stock Levels total; zero-cost items contribute 0 (see IV-041)	S2
IV-004	FUNC	Low-stock & expiry counts	Compare the "low stock" and "expiry risk" counts to Stock Levels / Expiry Risk	Counts equal the underlying lists	S3
IV-005	FUNC	Quick links	Click each tile/link	Navigates to the correct screen	S3
IV-006	UI	Empty state	New tenant, no products/stock	Cards show 0; nothing breaks; no NaN/₹NaN	S3

---

8. PRODUCT CATALOG & CATALOGUES

Screens: `FindProduct` (list/create), `EditProduct?productId=`, `CreateCatalogue`. Products are `mantle.product.Product` extended with `mrpAmount`, `hsnCode` / `sacCode`, `uqcEnumId` (unit), `gstSlabId` / `isGstExempt`, `enableBatchTracking`, `enableSerialTracking`, `reorderPoint` / `reorderQuantity`, `minimum/maximumStockLevel`, `isPerishable`, `inventoryTracked`. Org field: `ownerPartyId`.

8.1 Products

ID	Type	Title	Steps	Expected result	Sev
IV-010	FUNC	Create product	FindProduct → New → name, code (pseudold), type, HSN, unit (UQC), MRP, GST slab → Save	Product created under **your** org (`ownerPartyId`); opens EditProduct	S2
IV-011	FUNC	Create with opening stock	On create, enter an **opening stock** qty	Stock initialised to that qty; product becomes **tracked** (`inventoryTracked=Y`); appears in Stock Levels	S2
IV-012	FUNC	Create with opening stock = 0 / blank	Leave opening stock empty	Product created but **not tracked**; POS sales will **not** deplete it (by design — see IV-072)	S3

ID	Type	Title	Steps	Expected result	Sev
IV-013	FUNC	Edit product	EditProduct → change MRP / HSN / reorder point / tracking flags → Save	Persists; reflected in Stock Levels & dashboard	S2
IV-013a	FUNC	**Delete product (guarded)**	EditProduct → **Delete Product** (confirmation)	**[ZE-POS-014]** An unreferenced product (no bills/invoices/orders, no stock or movements) is permanently deleted with its prices, barcodes, category memberships and mode prices; a product on **POS bills / invoices / orders / stock records is BLOCKED** with "Cannot delete: … must stay for history/audit" — master data with history is never removed. Org-guarded (ownerPartyId, ADMIN exempt)	S2
IV-014	FUNC	Barcode / code search	FindProduct → scan or type a `pseudoId`	The matching product is found	S3
IV-015	FUNC	Stock-by-facility on product	Open a product with stock in ≥2 facilities	EditProduct shows QOH/ATP/value **per facility**	S3
IV-016	EDGE	Duplicate code	Create two products with the same `pseudoId`	Record actual — verify whether a duplicate code is blocked or allowed (flag if it collides with barcode scanning)	S3
IV-017	EDGE	GST-exempt product	Set `isGstExempt = Y` (Bill of Supply item)	No GST applied downstream; HSN still allowed	S3
IV-018	EDGE	Negative / zero MRP	MRP = −10 or 0	Record actual — verify validation; zero MRP → valuation 0 (see IV-041)	S3
IV-019	SEC	XSS in product name/description	Save `<img src=x onerror=alert(1)>` as the name	Rendered as text everywhere (list, edit, POS grid, PDFs); never executes	S1

8.2 Catalogues (categories)

ID	Type	Title	Steps	Expected result	Sev
IV-020	FUNC	Create catalogue	CreateCatalogue → name → Save	Category created, owned by **your** org (`ownerPartyId`)	S3
IV-021	FUNC	Add / remove product	Add a product to the catalogue, then remove it	Membership added (with fromDate) / date-ended on remove	S3
IV-022	ISO	Catalogue org lock	As a tenant, confirm you can only add **your** products to **your** catalogues	Guard blocks touching another org's catalogue/products	S1

---

9. WAREHOUSES & FACILITIES

Screens: `FindFacility` , `EditFacility?facilityId=` . Facility is `mantle.facility.Facility` extended with `taxRegistrationNumber` (GSTIN), `regionGeoId` / `gstStateCode` , `postalCode` . Type: Warehouse / Retail Store / Plant / Office. **The facility GSTIN/state drives inter-state IGST + e-Way Bill on transfers.**

ID	Type	Title	Steps	Expected result	Sev
IV-030	FUNC	Create facility	FindFacility → New → name, type, GST state code, GSTIN, pincode → Save	Facility created for your org	S2
IV-031	FUNC	Two same-state facilities	Create `WH_A` , `WH_B` with the **same** state code	Later transfers between them are **intra-state** (no IGST)	S3
IV-032	FUNC	Different-state facility	Create `WH_OTHER` with a **different** state code + GSTIN	Later transfers to it are **inter-state** (IGST + e-Way Bill)	S2
IV-033	FUNC	Facility stock breakdown	Open a facility with stock	EditFacility shows total value + per-product QOH/ATP/value	S3
IV-034	EDGE	Invalid GSTIN format	GSTIN = `123`	Record actual — verify format validation (15-char GSTIN); flag if it accepts junk that later prints on an e-Way Bill	S3

ID	Type	Title	Steps	Expected result	Sev
IV-035	ISO	Facility isolation	⚠ Facilities rely on **Mantle-level** owner scoping (no explicit org econdition on the screen). As tenant A, list facilities and try `EditFacility?facilityId=<B's facility>`	A sees only A's facilities; B's facility is **not** viewable	S1

10. OPENING STOCK & STOCK LEVELS

Screen: `StockLevels` (`StockByFacilityView` : `pseudoId` , `productName` , `facilityName` , `totalQoh` on-hand, `totalAtp` available-to-promise, `mrpAmount` , `totalValue` , `hsnCode`). Opening stock is entered at product-create (IV-011) or via a **Stock Adjustment of type "Opening"** (§12). Setting opening stock also sets an org-level `INVENTORY_TRACKING_ENABLED` flag so POS sales deplete stock.

ID	Type	Title	Steps	Expected result	Sev
IV-040	FUNC	Stock levels list	Open StockLevels	Every stocked product/facility row with QOH, ATP, value; org-scoped	S2
IV-041	CALC	Value = qty × cost	Pick a row	`totalValue` = on-hand × acquire cost (**not** MRP). A zero-cost item shows value **0** even with qty > 0 — **record** this and confirm it's expected	S2
IV-042	FUNC	Facility filter / search	Filter by facility / product	List narrows correctly	S3
IV-043	FUNC	Export	CSV / XLSX buttons	Export contains the same org-scoped rows	S3
IV-044	CALC	Opening stock reflects	Enter opening 100 for `PROD_TRACK`	StockLevels shows QOH 100 & ATP 100; product now `inventoryTracked=Y`	S2
IV-045	EDGE	ATP vs QOH	After a POS sale/reservation	ATP tracks QOH (this build moves them together); confirm no drift	S3

11. RECEIVING (Purchase → stock)

Screens: `FindReceipt` (approved/completed purchase orders), `ReceiveInventory?orderId=` . Receiving a PO line **creates an** `Asset` directly (batch): `statusId=AstAvailable` (immediately sellable — **no QC quarantine**), `quantityOnHandTotal` / `availableToPromiseTotal` = received qty, `acquireCost` from the PO, plus optional `batchNumber` , `manufacturingDate` , `expiryDate` , `mrpAmount` . An `AssetDetail` ledger row is written.

ID	Type	Title	Steps	Expected result	Sev
IV-050	FUNC	See receivable POs	FindReceipt	Lists **your** org's approved/completed purchase orders (`customerPartyId` = your org)	S2
IV-051	FUNC	Receive full qty	ReceiveInventory → enter qty accepted = ordered, pick facility → Receive	Asset created; StockLevels +qty at that facility; PO line shows fully received	S2
IV-052	FUNC	Receive with batch	Enter batchNumber, mfg date, expiry, batch MRP	Batch fields stored on the Asset; visible in Expiry Risk if expiry ≤ ~90d	S2
IV-053	FUNC	Partial receipt	Receive less than ordered; note "pending"	`qtyRemaining` = ordered − received; can receive the balance later	S2
IV-054	EDGE	Over-receipt	Receive **more** than ordered	Record actual — verify whether over-receipt is blocked or allowed (no cap in service)	S3
IV-055	CALC	Acquire cost carried	Receive at the PO unit price	`Asset.acquireCost` = PO unit price → StockLevels value uses this, not MRP	S2
IV-056	EDGE	No cost on PO line	Receive a line with no unit amount	Asset cost defaults to 0 → zero valuation (see IV-041); flag if unexpected	S3
IV-057	EDGE	Expiry before mfg date	Enter `expiryDate < manufacturingDate`	Record actual — **no** validation exists; flag the nonsensical data being accepted	S3

ID	Type	Title	Steps	Expected result	Sev
IV-058	FUNC	No QC quarantine	Receive goods	Stock is **immediately Available** (QC quarantine/approval services are stubs, not wired) — confirm goods are sellable at once; note if QC is expected	S3
IV-059	ISO	Receive cross-tenant PO	As tenant A, open `ReceiveInventory?orderId=<B's PO>`	Blocked / not found	S1

12. STOCK ADJUSTMENTS

Screens: `StockAdjustments` (list/create), `EditStockAdjustment?stockAdjustmentId=`. An adjustment has a **type** (PhysicalCount / Damage / Expiry / WriteOff / **Opening**) and a status flow **AdjPending → (Approve) → AdjApproved → (Apply) → AdjApplied**. Each item stores `systemQuantity`, `actualQuantity`, and `differenceQuantity = actual − system`. **Apply** calls `adjust#AssetOnHand(±diff)` per item and sets the product to `inventoryTracked=Y`.

ID	Type	Title	Steps	Expected result	Sev
IV-060	FUNC	Create adjustment	StockAdjustments → New → facility, type, remarks → Save	Created as **AdjPending**; `adjustedBy` = you; org-scoped	S2
IV-061	FUNC	Add item + diff	EditStockAdjustment → add product, enter **actual** count	System qty auto-filled from Asset; **difference = actual − system** shown (green gain / red loss)	S2
IV-062	CALC	Apply moves on-hand	Product system=100, actual=90 → Apply	On-hand becomes **90** (diff −10); AssetDetail ledger row written; status **AdjApplied**	S1
IV-063	CALC	Positive adjustment	actual=120 vs system=100 → Apply	On-hand **120** (+20)	S2
IV-064	FUNC	Opening via adjustment	Type = **Opening**, actual = starting qty on a new product	On-hand set; product becomes tracked	S3
IV-065	EDGE	Double-apply guard	Apply, then click Apply again	Second apply is a **no-op** (guard: status already AdjApplied); no double movement	S2
IV-066	EDGE	Apply with no items	Create an adjustment, add nothing, Apply	Blocked — "must have at least one item" (record actual)	S3
IV-067	EDGE	Negative result	actual = −10 (or actual far below system) → Apply	⚠ **No negative-stock guard** — on-hand can go negative. **Record actual** and flag (S3)	S3
IV-068	EDGE	Edit after apply	Try to add/remove items on an AdjApplied adjustment	Blocked in the UI (workflow guard)	S3
IV-069	FUNC	Tracking side-effect	Apply an adjustment for a previously **untracked** product	Product silently becomes `inventoryTracked=Y` → **from now on POS sales deplete it**. Confirm this is intended; note the side-effect	S3
IV-06A	ISO	Cross-tenant adjustment	As A, open `EditStockAdjustment?stockAdjustmentId=<B's>`	Blocked / not found	S1

13. STOCK TRANSFERS (inter-branch)

Screens: `StockTransfers` (list/create), `EditStockTransfer?stockTransferId=`. Status flow **StxDraft → (Ship) → StxInTransit → (Receive) → StxReceived** (+ Cancel from Draft). **Ship** calls `adjust#AssetOnHand(-qty)` on the **from** facility; **Receive** calls `adjust#AssetOnHand(+qty)` on the **to** facility. If the two facilities have **different GSTINs**, an **IGST branch-transfer invoice** is created and `ewayBillRequired=Y`; also auto-`Y` when `totalValue > ₹50,000`. E-Way Bill requires `transporterGstin` + `vehicleNumber` **at ship time**.

ID	Type	Title	Steps	Expected result	Sev
IV-070	FUNC	Create transfer	StockTransfers → New → from `WH_A`, to `WH_B`, date → Save	Transfer created as **StxDraft**, org-scoped	S2
IV-071	FUNC	Add items	EditStockTransfer (Draft) → add product + qty (+ batch if tracked)	Item added; `unitAmount` defaults to product MRP	S2

ID	Type	Title	Steps	Expected result	Sev
IV-072	CALC	Ship depletes source	Ship a 20-unit transfer from `WH_A` (had 77)	`WH_A` QOH → **57**; status **StxInTransit**; source AssetDetail −20	S1
IV-073	CALC	Receive adds to dest	Receive the transfer at `WH_B` (had 0)	`WH_B` QOH → **20**; status **StxReceived**; dest AssetDetail +20; **net org stock unchanged**	S1
IV-074	FUNC	Intra-state, no IGST	Transfer `WH_A → WH_B` (same GSTIN)	No IGST invoice; `ewayBillRequired` only if value > ₹50k	S2
IV-075	CALC	Inter-state IGST + e-Way	Transfer `WH_A → WH_OTHER` (different GSTIN)	`ewayBillRequired=Y` ; an **IGST branch-transfer invoice** (`igstInvoiceId`) is created	S1
IV-076	FUNC	e-Way Bill gate at ship	For an e-Way-required transfer, Ship **without** transporter GSTIN / vehicle no.	Blocked with an **error message** (soft — the screen must show it, not silently proceed). Record actual	S2
IV-077	CALC	>₹50k auto e-Way	Same-state transfer with total value > ₹50,000	`ewayBillRequired` auto-set **Y** even intra-state	S2
IV-078	EDGE	Ship more than on-hand	Item qty 150 when `WH_A` has 100 → Ship	⚠ **No source-stock guard** — `WH_A` goes to **−50**. Record actual & flag (S3)	S3
IV-079	EDGE	Same-facility transfer	from `WH_A` , to `WH_A` → Ship → Receive	Record actual — no-op (−then+ same facility); verify whether it's blocked/warned (S3)	S3
IV-080	EDGE	Partial receive	Ship 100; receive **95** (enter quantityReceived)	Dest +95; **5 units unaccounted** (no shrinkage tracking). Record & flag	S3
IV-081	EDGE	Over-receive	Ship 100; receive **125**	Record actual — no cap; verify	S3
IV-082	EDGE	Double-ship / double-receive	Click Ship twice; Receive twice	Status guard should prevent a second movement (must **not** double-deplete/double-add). **Critical** if it does	S1
IV-083	EDGE	Cancel draft	Cancel a Draft transfer	Status Cancelled; **no** stock moved	S3
IV-084	EDGE	Batch on transfer	Transfer a **batch-tracked** product without a batch number	Record actual — allowed (batch may be lost); flag if traceability breaks	S3
IV-085	ISO	Cross-tenant transfer	As A, open `EditStockTransfer?stockTransferId=<B's>`	Blocked / not found	S1

14. BATCH & SERIAL TRACKING

Batch: on the `Asset` (`batchNumber` , `expiryDate` , `manufacturingDate` , `mrpAmount`). **Serial:** `ProductSerialNumber` (unique `serialNumber` , `facilityId` , `statusId` , warranty date, PO/SO links, `batchNumber`) with a lifecycle: **SnInStock ⇄ SnReserved → SnSold → SnReturned → (SnInStock | SnDefective) → SnScrapped**, plus **SnInStock ⇄ SnInWarranty**. Screen: `SerialNumbers` (register + summary metrics + `SerialNumberHistory` audit).

ID	Type	Title	Steps	Expected result	Sev
IV-090	FUNC	Register serial	SerialNumbers → register (product with serial tracking, serial no., facility)	Serial created as **SnInStock**; appears in the list; summary counts update	S3
IV-091	EDGE	Duplicate serial	Register the **same** serial number twice	Blocked / rejected (serials must be unique). Record actual	S2
IV-092	EDGE	Serial on non-serial product	Register a serial for a product with `enableSerialTracking≠Y`	Blocked (record actual)	S3
IV-093	FUNC	Lifecycle transitions	Move a serial InStock→Reserved→Sold→Returned→Restock/Defective→Scrapped	Each valid transition succeeds; invalid ones blocked; **SerialNumberHistory** row written per change	S3
IV-094	EDGE	Serial vs stock transfer	Move a batch of a serial-tracked product via a **Stock Transfer**	⚠ Known gap — the transfer moves the Asset qty but **does not update the serial's** `facilityId` → serial/location mismatch. Record & flag	S3

ID	Type	Title	Steps	Expected result	Sev
IV-095	FUNC	Warranty expiry field	Register with a `warrantyExpiryDate`	Stored; note it is **separate** from batch `expiryDate` (not reconciled)	S4
IV-096	ISO	Serial isolation	As A, list serials	Only A's serials (`ProductSerialNumberView.ownerPartyId`)	S1

15. EXPIRY RISK

Screen: `ExpiryDashboard` . Lists batches whose `Asset.expiryDate` is within ~90 days, with batch no., expiry, quantity and value; totals the value at risk.

ID	Type	Title	Steps	Expected result	Sev
IV-100	FUNC	Expiry list	Receive/adjust a batch with expiry in ~30 days; open Expiry Risk	The batch appears with days-to-expiry, qty and value	S3
IV-101	CALC	Value at risk	Compare the total	Total = Σ(expiring qty × valuation rate); zero-cost batch → 0	S3
IV-102	EDGE	Already-expired stock	A batch with `expiryDate` in the **past**, qty > 0	Shows in the list; ⚠ expired stock is **not blocked** from sale/transfer elsewhere — note this	S3
IV-103	**ISO**	**Cross-tenant expiry leak**	⚠ **Suspected gap.** As tenant A, open Expiry Risk while tenant B has expiring batches	**MUST show only A's batches.** If B's expiring stock/values appear → **Critical cross-tenant leak (S1)** — capture evidence	S1
IV-104	UI	Empty state	Tenant with no near-expiry stock	Clean "nothing expiring" state; no error	S4

16. DEMAND FORECAST & REORDER ALERTS

Screen: `DemandForecast` . "Run Cycle" triggers `cycle#DemandForecast` (per-org): calibrates past forecasts against actual POS sales, generates new forecasts, detects **demand-shift anomalies** (z-score ≥ 2σ), and raises **reorder alerts**. Reorder point **ROP = avgDailyDemand × leadTime + safetyStock**; **safetyStock = Z(serviceLevel) × σ × √leadTime** (Z≈1.28 @90%, 1.645 @95%, 2.33 @99%). Alerts (`DemandAlert`): `AlertNew` → `AlertAck` .

ID	Type	Title	Steps	Expected result	Sev
IV-110	FUNC	Run cycle	On a throwaway tenant **with POS sales history**, click **Run Cycle**	Forecasts generated; alert list populates; metrics (active forecasts, avg accuracy) update	S3
IV-111	CALC	Reorder alert fires	For a tracked product with SOH ≤ ROP	A **Reorder** alert appears; SOH ≤ safetyStock → **Critical**; SOH ≤ ROP×1.2 → **LowStock**	S3
IV-112	CALC	ROP / safety math	Spot-check one product's forecast	ROP ≈ avgDaily×leadTime + Z×σ×√leadTime (verify with a calculator)	S3
IV-113	EDGE	No sales history	Run for a brand-new product (no POS sales)	**No** forecast/reorder alert generated (needs ≥ ~2 weeks / ≥2 sale dates); no crash	S3
IV-114	EDGE	Untracked product	A product without opening stock but with sales	Reorder alert only for **tracked** products — untracked ones are skipped (avoids false "reorder everything")	S3
IV-115	FUNC	Demand-shift alert	Create a spike in recent 7-day sales vs baseline	A **Shift/Increase** alert (z ≥ 2σ) is raised, deduped per day	S3
IV-116	FUNC	Acknowledge alert	Click acknowledge on an alert	Status → `AlertAck` ; drops off the "new" list	S3
IV-117	PERF	Notifications	After Run Cycle	Alerts push a notification to org users — confirm this fires **only** on the throwaway tenant (see rule #7)	S3
IV-118	ISO	Forecast/alerts scoped	View as two tenants	Each sees only its own forecasts/alerts (`organizationPartyId`)	S1

17. GATE PASSES

Screen: `GatePasses`. Type: **Returnable (GptRgp)** / **Non-Returnable (GptNrgp)**. Status: **GpIssued → GpReturned → GpClosed**, or **GpIssued → GpOverdue**. Fields: facility, party, issue/expected-return dates, vehicle, transporter, items with `quantity` / `returnedQuantity`.

ID	Type	Title	Steps	Expected result	Sev
IV-120	FUNC	Issue RGP	GatePasses → New → Returnable, facility, party, expected-return date, items → Save	Gate pass created as **GpIssued**, org-scoped	S3
IV-121	FUNC	Issue NRGP	Non-Returnable pass	Created; no expected-return date required	S3
IV-122	FUNC	Return / close	Record returned quantities; close the pass	Status → GpReturned/GpClosed; returned qty recorded	S3
IV-123	EDGE	Overdue	An RGP past its expected-return date	Verify it can be marked/shown **GpOverdue**	S3
IV-124	EDGE	Gate pass vs stock	Confirm whether issuing a gate pass moves on-hand or is documentation-only	Record actual behaviour (does RGP reduce/reserve stock?) and note it	S3
IV-125	ISO	Gate pass scoped	Two tenants	Each sees only its own passes (`organizationPartyId`)	S1

---

18. INTEGRATION — stock movement end-to-end

The real test of inventory is whether stock moves correctly when **other modules** act. Run these on the throwaway tenant.

ID	Type	Title	Steps	Expected result	Sev
IV-130	CALC	POS sale depletes	`PROD_TRACK` QOH 100 → sell **3** at the POS Terminal → complete the bill	QOH → **97** at the org's default facility; AssetDetail row "POS Sale "	S1
IV-131	EDGE	Untracked not depleted	Sell `PROD_UNTRACK` (no opening stock) at POS	Stock **unchanged** (only `inventoryTracked=Y` products deplete) — confirm this is intended	S2
IV-132	CALC	Purchase receipt adds	Receive a PO for `PROD_TRACK` (+50)	QOH +50; value uses PO cost	S2
IV-133	CALC	Full loop	Receive 50 → transfer 20 to `WH_B` → sell 3 at POS → adjust −1 for damage	Net numbers reconcile: source/dest QOH and total org on-hand all add up; each step has a ledger row	S1
IV-134	EDGE	Oversell below zero	Sell more than on-hand for a tracked product at POS	⚠ No negative guard in the stock service — record whether POS itself blocks it or stock goes negative; **flag**	S2
IV-135	CALC	POS backlog sync	Complete a POS bill, then check Stock Levels shortly after	Depletion appears (POS→invoice→stock runs async / backlog cron) — confirm within a moment, not instantly	S3

---

19. MULTI-TENANT ISOLATION (high priority)

ID	Type	Title	Steps	Expected result	Sev
ISO-001	ISO	Products scoped	Tenant A lists Product Catalog	Only A's products (`ownerPartyId`)	S1
ISO-002	ISO	Stock levels scoped	A opens Stock Levels	Only A's stock rows	S1
ISO-003	ISO	Transfers/Adjustments/GatePasses scoped	A lists each	Only A's records (`organizationPartyId`)	S1
ISO-004	ISO	Direct-URL edit	A opens B's `EditProduct` / `EditFacility` / `EditStockTransfer` / `EditStockAdjustment` / `ReceiveInventory` by id	Blocked / not found for every one	S1

ID	Type	Title	Steps	Expected result	Sev
ISO-005	**ISO**	**Expiry Risk leak**	A opens `ExpiryDashboard` while B has expiring stock	Only A's batches (see IV-103) — **verify hard**	S1
ISO-006	ISO	Serials/Forecast/Alerts scoped	A views each	Only A's rows	S1
ISO-007	ISO	Pickers scoped	Product/facility dropdowns in any create screen	Only A's products/facilities	S1
ISO-008	ISO	No cross-tenant **write**	(Dev-assisted) attempt an adjustment/transfer/receipt referencing B's product/facility while logged in as A	Rejected — a tenant must not mutate another org's stock	S1

20. UI / ACCESSIBILITY / SECURITY

ID	Type	Title	Check	Expected result	Sev
IV-140	UI	Desktop layout	All screens at 1920 / 1366	Grids, badges and totals aligned; no overflow; status colours correct (gain green / loss red)	S4
IV-141	UI	Mobile layout	Create/receive/adjust at 360–390px	No horizontal scroll; forms usable; numbers readable	S3
IV-142	A11Y	Keyboard	Tab through a create/receive/adjust form	Logical order; Enter submits; visible focus	S3
IV-143	SEC	XSS in free text	Put `<img src=x onerror=alert(1)>` in product name / remarks / batch no. / gate-pass party	Escaped as text everywhere (lists, dashboards, exports); never executes	S1
IV-144	SEC	CSRF	(Dev-assisted) POST a create/ship/apply without the session token	Rejected	S2
IV-145	EDGE	Double-submit	Click Save / Ship / Receive / Apply twice fast	Exactly **one** effect (no double stock movement)	S1
IV-146	PERF	Large catalog	Tenant with thousands of products/assets	Stock Levels & dashboard load in reasonable time; export completes	S3

21. Cross-flow / regression & defect template

ID	Type	Title	Expected result	Sev
XF-001	CALC	Receive → Transfer → Sell → Adjust	Every quantity reconciles across facilities and against the ledger (IV-133)	S1
XF-002	CALC	Inter-state transfer	Correct IGST invoice + e-Way Bill; source − / dest + exactly; net org stock unchanged	S1
XF-003	ISO	End-to-end isolation	Nothing tenant A creates (product/stock/transfer/serial/alert/expiry) is visible or mutable by B	S1
XF-004	EDGE	Negative-stock sweep	Force negative via transfer, adjustment and oversell; document each result	S2
XF-005	CALC	Tracking flag lifecycle	Untracked → apply adjustment → becomes tracked → POS now depletes; confirm the transition point	S2

```
Defect ID:        ZE-INV-____
Linked Case ID:   IV-0__ / ISO-0__ / XF-0__
Title / Severity / Type:
Environment:      Prod | <browser> | tenant: <org>
Steps / Expected / Actual:   (for CALC cases, show the arithmetic: before qty, movement, after qty, value)
Product / Facility / Asset / Transfer / Adjustment ids touched:
Evidence:         screenshot + Console/Network (mandatory for CALC, ISO, SEC)
```

---

22. Areas to probe hardest (attention map)

These are the most likely defects in this module — spend extra time here:

1. **Expiry Risk cross-tenant isolation (IV-103 / ISO-005)** — *fixed*: the dashboard now scopes the Asset query by `ownerPartyId`, and `Asset` was added to the active-org entity-filter set (`ZTF_ASSET`) so **every** raw Asset find is org-scoped. Re-test with two tenants to confirm no leak.
2. **No negative-stock guard (IV-067, IV-078, IV-134)** — *partly fixed*: transfer **ship** now refuses to move more of a *tracked* product than the source holds; **adjustment** rejects a negative counted quantity. POS oversell is intentionally **not** blocked at the stock layer (a sale shouldn't be refused on a count) — reconcile via adjustment.
3. **Double-movement on ship/receive/apply (IV-082, IV-145)** — *fixed*: `for-update` row locks on transfer ship/receive and adjustment apply serialize concurrent clicks so the same movement can't post twice.
4. **The `inventoryTracked` flag (IV-012, IV-069, IV-114, IV-131, XF-005)** — a product only deplete-on-sale once tracked; adjustments/opening flip it on for **all facilities** (intended: entering a count = you now track that product). Confirm the flag's lifecycle and its POS side-effects.
5. **Inter-state transfer IGST + e-Way Bill (IV-075, IV-076, IV-077, XF-002)** — *e-Way gate fixed* (now a hard block if transporter GSTIN / vehicle no. are missing). Still verify the **IGST branch-transfer invoice** is correct for a different-GSTIN transfer.
6. **Valuation with zero cost (IV-041, IV-056, IV-101)** — no MRP/acquire cost → stock valued at 0. Confirm whether that's acceptable or a data-quality trap.
7. **Batch/serial traceability (IV-084, IV-094)** — *fixed*: a batch-tracked transfer now moves from one source lot and **recreates that lot (with its expiry) at the destination**; serial-tracked transfers **relocate the moved serials** to the destination facility with a history audit row. Re-test that a lot's expiry still shows in Expiry Risk after a transfer, and that a moved serial reports the new facility.
8. **Partial/over receipts & transfers (IV-053, IV-054, IV-080, IV-081)** — no shrinkage/overage reconciliation; missing units simply vanish.
9. **Not-yet-wired features** — **QC quarantine/approval** and **WMS pick-list** services are **mock stubs** (received goods go straight to Available). Don't test them as live; note them as gaps.
10. **Facility isolation (IV-035)** — facilities rely on Mantle-level owner scoping with no explicit screen econdition; confirm a tenant genuinely cannot open another org's facility by id.

---

23. Teardown (mandatory)

1. **List every test artefact** — products, facilities, catalogues, opening-stock/adjustments, transfers, receipts, serials, gate passes, forecasts/alerts — with ids.
2. **Reverse stock movements** you introduced on throwaway data with an **equal-and-opposite** adjustment (applied adjustments and shipped transfers can't be un-applied). Bring test facilities back to zero on-hand where possible.
3. **Cancel** draft transfers/adjustments; **close** gate passes; **acknowledge/clear** test alerts so they don't re-notify.
4. Delete any **exports/PDFs** generated on the test machine.
5. Confirm you **never mutated stock on MARUTHI** (org `100000`) or any live tenant, and that the **Demand Forecast Run Cycle** was only ever run on a throwaway tenant.
6. Attach the run sheet(s) + defect ids to this part's report.

---

*End of Part 5 — Inventory. Related: Part 3 — POS (counter billing depletes stock); Part 4 — Sales; Purchase (receipts add stock) and Accounting (valuation) are separate parts. The transfer form now has per-line **lot** and **serial-number** pickers (choose exactly which batch / which units to move; blank = the ship auto-picks the earliest-expiry lot and Receive auto-relocates the first N serials). Remaining known gaps to triage with the dev: QC quarantine/approval and the WMS pick-list are mock stubs (not wired).*